Algorithms – Final Exam

8.3.26 – term B

Prof. Tami Tamir

- Exam length: 3 hours
- Allowed open material: Attached fact-sheets + one paper brought by the student.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- You may use scratch paper, but answers must be written only on the exam form.
- Note that only the front side of each page is scanned for grading. If you need more space, use the last page and clearly direct the grader there.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

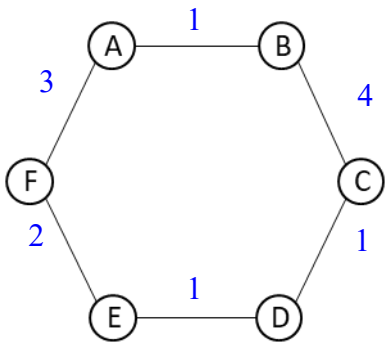
G O O D L U C K !

Question 1 (18 pts.)

♣ **1.1 (3*3=9 pts.)** Consider the cycle graph C_6 in the figure. The graph is an input to the minimum spanning tree problem. The edge weights are known to be 1,1,1,2,3,4. That is, three edges have weight 1and the remaining edges have weights 2,3,4. The assignment of these weights to the edges is not known.

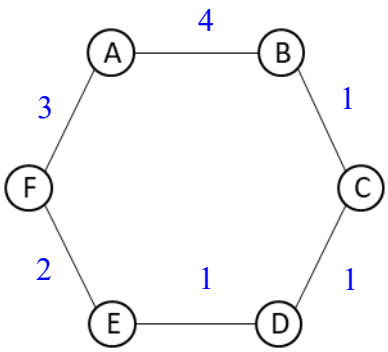
For each of the following sections (independently), specify edge weights next to the edges such that the described requirements are met, or briefly explain why no solution exists. If a solution exists, it may not be unique, and you do not need to justify your answer.

a. There exists a run of Kruskal in which the edge (AB) is the first to join the MST, and a run of Prim that begins at node C, in which the edge (AB) is the last to join the MST.



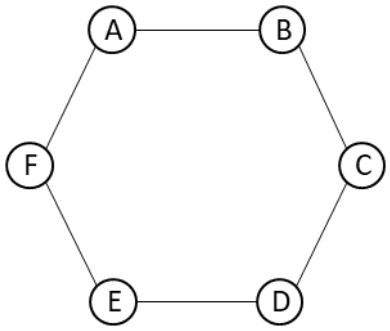
Any assignment in which w(AB)=1 and w((BC)=4.
Another variant: the costs, starting with AB in clockwise direction are (1,3,2,1,1,4)

b. In every run of Kruskal, and in every run of Prim that begins at node C, at some point, the connected components are {A}, {B,C,D,E}, {F}.



Any assignment in which w(BC)=w(CD)=w(ED)=1

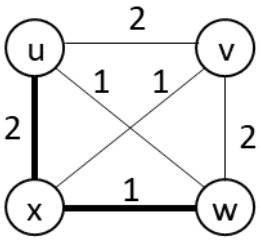
c. There exists a run of Kruskal in which the edge (CD) is the last to join the MST, and a run of Prim that begins at node C, in which the edge (AB) is the third to join the MST.



No such assignment exists. By the first condition w(CD)=3. If w(BC)=4, then in a run of Prim, the edges will be added in clockwise direction. If w(BC)<4 then it must be that both AB and CB have weight at most 2, and they are the first two edges to join the MST.

♣ **1.2 (9 pts.)** Let G=(V,E) be a simple, connected, undirected graph with positive edge weights, $w: E \to \mathbb{N}$. Given is a set of edges $A \subseteq E$. Suggest an efficient algorithm that returns a spanning tree of G that contains the maximum possible number of edges from A, and has minimum weight. That is, a spanning tree that includes as many edges from A as possible, and whose weight is the smallest among all spanning trees containing that maximum number of edges from A.

For example, in the given graph, for A={(u,x),(x,w)}, the solution is a tree containing the edges {(u,x),(x,w),(x,v)} with a total weight of 4. Note that the graph has a Minimum Spanning Tree (MST) with a weight of 3, but it contains only one edge from A.



The time complexity of your algorithm should be at most O(|E|log|E|).
Describe the algorithm and analyze its time complexity.
No need to justify or prove correctness.

Algorithm:

1. Let C be a constant such that for all $e \in A$ it holds that $w(e) < C$.
2. Define a new weight function w' for each edge $e \in E$:
   For every edge $e \in A$, define $w'(e) = w(e)$, For every edge $e \notin A$, define $w'(e) = w(e)+C$.
3. Run Kruskal's Algorithm (or Prim's Algorithm) using w' as weights

Time Complexity:

1. Assigning the new weights takes O(|E|).
2. Running an MST algorithm (say, Kruskal): O(|E| log |E|).

Total: O(|E| log |E|).

Proof idea (not required): By shifting the weights of edges not in A by a big constant, the algorithm is forced to prioritize edges from A. It adds as many as possible such edges without forming a cycle. Only when it is impossible to add more edges from A, the algorithm considers edges from E-A. Within each group, the original weights w(e) ensure that the lightest possible edges are chosen to complete the spanning tree.

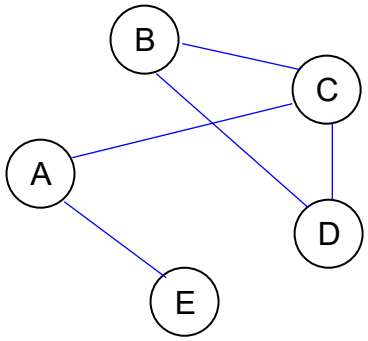
An alternative way to describe this algorithm: Let L be a list consisting of the edges of A sorted in non-decreasing order followed by the list of E-A sorted in non-decreasing order.
Starting from an empty graph, consider the edges in L, add any edge that doesn't close a cycle.

Question 2 (12= 2*6 points)

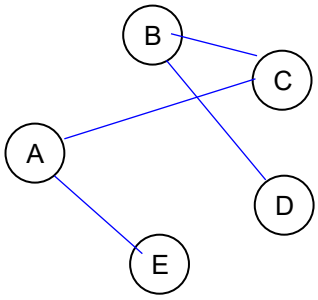
In both sections assume that the graphs are represented by adjacency lists in lexicographical order, and any tie-breaking is resolved according to lexicographic order.

♣ **2 1.** Complete the figure such that the result is a simple **undirected** graph, G, over V={A,B,C,D,E} that satisfies the following three conditions.

a. When running DFS(A), the first edge added to the tree is (A,C), and in the resulting undirected tree, the degree of A is 2 and the degree of B is 2.
b. G has a Hamilton path.
c. G has an Euler path consisting of 5 edges, that begins at C and ends at E.



- Complete the figure with the DFS tree corresponding to condition a:



- A Hamilton path corresponding to condition b is:

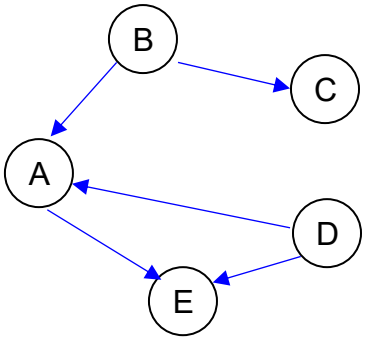
EACBD

- An Euler path corresponding to condition c is:

CBDCAE

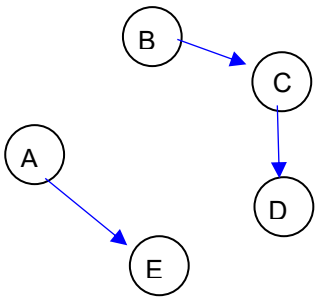
♣ **2.2** Complete the figure such that the result is a simple **directed** graph, G, over V={A,B,C,D,E} that satisfies the following three conditions.

a. Running DFS(A) results in a forest containing two trees, one with 2 nodes, and one with three nodes.
b. Running DFS(B) results in a forest containing one tree.
c. G has a unique topological sort.

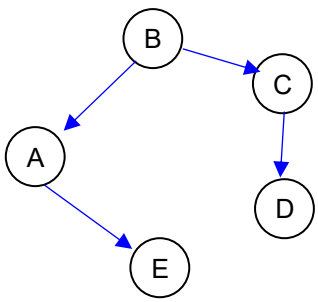


Another solution: G is the directed path BCAED

Draw the DFS(A) forest corresponding to condition a



Draw the DFS(B) forest corresponding to condition b



Write the topological sort of G

BCDAE

Question 3 (18 points)

An avocado farmer sells her produce in two markets: the **North Market** and the **South Market**. The selling lasts n days. In each day, she can sell in at most one market. For each day, $1 \le i \le n$, the expected profit from selling in the North Market is Ni$, and the expected profit from selling in the South Market is Si$. All values are positive and known in advance. The logistics of transporting the produce are complex: if the farmer decides to move her operations from one market to another, she must dedicate an entire day to transportation, unloading, and reorganization. Therefore, on a day when switching markets, nothing is sold. For example, if she sold in the North Market on the fifth day and wishes to sell in the South Market on the seventh day, then the sixth day must be a transport day where the profit is 0.

Suggest an algorithm based on **dynamic programming** that receives as input the expected profits for each day in each market (2n values), and calculates the farmer's maximum possible total profit.

Let **R(i, North)** denote the maximum profit that can be accumulated in the first i days, given that on day i the produce is sold in the North Market. Similarly, let **R(i, South)** denote the same for the South Market.

3.1 (2 pts.) What is the expression describing the solution's value (no need to explain)?

Max(R(n, North), R(n, South))

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating **R(i, North)** and **R(i, South).** Distinguish between the base and the general case. Explain the formula in words.

Base case: For i = 1: R(1, North) = N1, R(1, South) = S1
For i = 2: R(2, North) = N1 + N2, R(2, South) = S1 + S2.
(Note that transitioning on day 2 is never optimal)

Alternatively, in the base case, instead of i=2, define R(0, North) =R(0,South)=0.

General Formula (for i > 2):
R(i, North) = Ni + Max(R(i-1, North), R(i-2, South))
R(i, South) = Si + Max(R(i-1, South), R(i-2, North))

Explanation: To be in the North market on day i, the seller either stayed there from the previous day (after achieving R(i-1, North)) or moved from the South market. If she moved, day i-1 was dedicated to transport (earning zero), meaning she was in the South market on day i-2.

♣ **3.3** (4 pts.) Describe the DP table, its size, and the order of filling. Analyze time complexity as a function of n. Justify your answer.

The table has size n*2 (or two arrays of size n). Assume the rows correspond to North and South and n columns to days 1 to n. The base case is the first two columns, we then fill column after column, the entries in column i depend on entries in columns i-1, i-2. Every entry takes time O(1) for a total of 2n*O(1)=O(n).

♣ Question 4 (14 points)

The City Council wishes to illuminate a long, straight walking path that starts at point 0 and ends at point L. Along the path, n poles are placed at locations $d_1, d_2, ..., d_n$. The locations are measured from the start of the path at point 0, such that $0 \le d_1 \le d_2 \le \ldots \le d_n \le L$. Lamps can only be installed on these existing poles.

Each lamp has a lighting radius of r meters in each direction. That is, if a lamp is installed on a pole at location x, it illuminates the entire segment between x-r and x+r. The path is considered fully illuminated if every point on the line between 0 and L is within the lighting range of at least one lamp. The goal is to determine the locations of the poles where lamps should be installed, such that the entire path is illuminated using the minimum number of lanterns.

Assumptions: $d_1 \le r$, $L-d_n \le r$, and for every $1 \le i \le n-1$, $d_{i+1}-d_i \le 2r$.
Note that these conditions guarantee that a valid solution to the problem exists.

Example: For a path of length L=100m, lighting radius r=20m, and pole locations $d_1=13$, $d_2=18$, $d_3=35$, $d_4=47$, $d_5=82$, $d_6=90$. An optimal solution includes three lamps placed on the following poles: at point d_2 (lighting the range [0, 38]), at point d_4 (lighting the range [27, 67]), and at point d_5 (lighting the range [62, 102], covering until the end of the path at 100). There are other solutions with three lamps, but it is impossible to cover the entire path with only two lamps.

Suggest a greedy algorithm to solve this problem. Describe the algorithm and analyze its running time. **State** and **prove** the Greedy Choice Property suitable for proving the correctness of your algorithm.

Idea: The algorithm proceeds from the start of the path (0) toward the end (L), greedily picking the furthest possible pole that still covers the current rightmost "dark" point.

Initialize: Covered = 0.
While Covered < L:
    Let i be the maximal such that $d_i - r \le$ Covered
    Install a lamp on the pole at point d_i.
    Update Covered = $d_i + r$.

Time complexity:
Init: O(1).
Loop: We traverse the list of poles $d_1, d_2, ..., d_n$ once. In each step, we advance on the list to see which is the furthest that satisfies the coverage condition. Since the poles are already sorted ($0 \le d_1 \le d_2 \le \ldots \le d_n \le L$), we can use a single pointer to move through the list.
Total Time Complexity: O(n)

GCP: There exists an optimal solution for the lighting problem that includes the pole at point d_k, where k is the maximal such that $d_k - r \le 0$

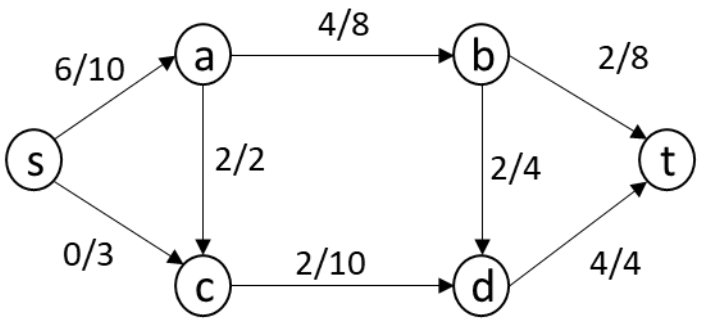
Proof: Let A be any solution to the problem. Let j be the index of the leftmost pole in A, and assume that $k \neq j$. The lamp installed at d_j must cover the start of the path at 0, therefore $d_j - r \le 0$. By the greedy choice, $d_k > d_j$. Consider the set of poles $A' = A \setminus \{j\} \cup \{k\}$. A lamp installed at d_j covers the range $[0, d_j+r]$. A lamp installed at d_k covers the range $[0, d_k+r]$. Since $d_k > d_j$, the exchange can only increase the covered range, therefore A' is a feasible solution such that $|A|=|A'|$ and A' includes d_k.

Solution to Question 4 (cont' – if needed)

Question 5 (20 points)

♣ **5.1** (**8 pts**.) The figure presents a flow network with a legal flow. Every edge $e \in E$ is labeled by f(e)/c(e) . It is known that the given flow is a result of **two** iterations of Ford & Fulkerson algorithm.

No need to justify your answers in this section.



1. What was the augmenting path in the first iteration? sabdt

2. What was the augmenting path in the second iteration? sacdbt

3. What is the value of the maximum flow in the network? 12

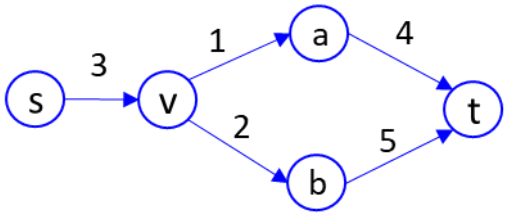
4. Write all sets of nodes that define a min-cut in the network:

{s,a,b,c,d}, {s,a,c,d}

5.2 (12 pts. = 3*4) Let G=(V,E) be a flow network with a capacity function $c: E \to \mathbb{N}$, a source node s, and a sink node t. Prove or disprove each of the following claims (separately):

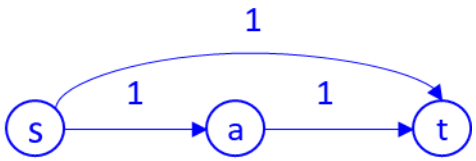
♣ **5.2.1** If the capacity function is injective (that is, for every e1 ≠ e2, it holds that c(e1) ≠ c(e2)), then there exists a unique minimum cut.

False.
Both {s} and {s,v} are min-cuts



♣ **5.2.2** If the network has more than one minimum cut, then it is impossible to increase the maximum flow in the network by increasing the capacity of a single edge.

False.
Both {s} and {s,a} are min-cuts.
The max-flow will increase if c(s,t) increases.



♣ **5.2.3** If all capacities in the network are integers that are multiples of 5, then the value of the maximum flow is an integer that is a multiple of 5.

Proof: The value of the max flow is defined as the sum of the capacities of out-edges in a min cut. This is a sum of multiples of 5.

♣ Question 6 (18 points)

An **undirected** graph G=(V,E) represents a network of routes in a national park. Constant winds blow in different directions across the park; therefore, for every edge e=(u,v) in the graph, two weights are given: w(u,v) represents the crossing time of the edge when walking from u to v, and w(v,u) represents the crossing time of the edge when walking from v to u. If $w(u,v) \le w(v,u)$, then walking from u to v is "with the wind", and walking from v to u is "against the wind."

Given a source node $s \in V$, suggest an efficient algorithm that finds, for every node $v \in V$, the duration of a fastest path from s to v under the constraint that the path includes at most one edge walked against the wind.
Describe the algorithm in words and formally, justify its correctness (state the main claim and justify it briefly), and analyze its time complexity.

1. Define a **directed** graph G'=(V',E'), $V'=V^1 \cup V^2$ where each of V^i is a copy of V.
For every undirected edge e=(u,v) in E, add to E' three directed edges:
Assume w.l.o.g., that $w(u,v) \le w(v,u)$
- Add (u^1,v^1) and (u^2,v^2) to E' both with weight w(u,v).
- Add (v^1,u^2) to E' with weight w(v,u)

In words: There are two copies of the vertices. For every edge (u, v) in E such that $w(u,v) \le w(v,u)$, we add to G' edges (u^1,v^1) and (u^2,v^2) with weight corresponding to 'with the wind' direction, and an edge (v1, u2) with weight corresponding to 'against the wind' direction.

2. Run Dijkstra on G' with s^1 as a source.
3. For every v in V, return $\min(d(s^1,v^1), d(s^1,v^2))$.

Correctness: In G'
- There is a path from s1 to v1 that has weight k, iff there is a path from s to v in G that only go with the wind direction, and has weight k.
- There is a path from s1to v2 that has weight k, iff there is a path from s to v in G that use one edge against the wind, and has weight k.

Main idea: The paths within each copy include only edges going with the wind direction. The transition from the first copy to the second is on a single edge against the wind.

Time complexity:
Step 1: O(|V|+|E|). G' has 2|V| nodes and 3|E| edges.
Step 2: O(|E'|+|V'|log|V'|) = O(|E|+|V|log|V|) assuming implementation of Dijkstra with Fib. heaps.
Step 3: O(|V|)

All together: O(|E|+|V|log|V|)